

Team Details

Vehicle Rental Management System

CIA-3 | Advanced JavaScript Backend Frameworks

Christ University (L&T EduTech)

Team members: [Add names and registration numbers]

Team roles: M1 Foundation | M2 Core Workflow | M3 Reporting and Access | M4
Infrastructure and Documentation

GitHub: <https://github.com/antiiiiny/vehicle-rental-management>

Submission date: 10 September 2026

Project Overview

The system provides a full rental workflow for cars and bikes: customers discover available vehicles, reserve a date range, and view rental history; branch staff record pickup and return inspections; administrators manage branches, fleet, pricing, and reports.

The backend is built with Node.js, Express, MongoDB, and Mongoose. JWT authentication, bcrypt password hashing, express-validator, role-based access control, and centralized JSON error handling protect the API.

Objectives and Requirements

1. Deliver secure registration and login with role-aware access.
2. Prevent double-booking with a standard date-interval overlap rule.
3. Support reservation, pickup, return, cancellation, damage, fuel, and late-fee workflows.
4. Provide pricing add-ons, customer history, utilization metrics, and financial summaries.
5. Offer a browser UI plus a complete Postman collection for demonstration and testing.

Module Summary

Foundation: authentication, branches, vehicles, availability search.

Core workflow: booking creation and conflict checks, status transitions, pickup and return inspections.

Reporting and access: add-on catalog, cancellation policy, rental history, utilization and financial reports, RBAC.

Integration: single-page frontend covering registration, search, booking, inspections, history, and admin analytics.

Architecture

Client: static HTML, CSS, and JavaScript in public/.

API: Express route modules under routes/ delegate to controllers/.

Domain: Mongoose models under models/ persist users, branches, vehicles, bookings, and inspections.

Cross-cutting concerns: auth.js verifies JWTs, rbac.js checks roles, validate.js handles express-validator results, and errorHandler.js normalizes failures.

Database: MongoDB with referenced ObjectIds and indexes for common lookup and overlap queries.

Data Model and Reference Decisions

User references Branch for branch staff ownership.

Vehicle references Branch because fleet assets belong to a branch but have their own lifecycle.

Booking references Vehicle and User because both are independently managed and reused across many bookings.

Inspection references Booking, Vehicle, and User because each inspection is an auditable event.

Booking embeds selected add-on snapshots so historical prices do not change when the catalog changes.

API Demonstration and Business Rules

Register or log in and copy the returned token into the Postman token variable.

Search availability with branchId, startDate, and endDate. The conflict condition is $\text{startDate} < \text{requestedEnd}$ AND $\text{endDate} > \text{requestedStart}$.

Create a booking with optional add-ons. The API calculates totalDays, base rate, add-ons, and total amount.

Record pickup, then return inspection. Return charges update finalAmount and move the booking to returned.

Cancellation fees are time-tiered: at least 48 hours is free, 24-48 hours charges 20%, and under 24 hours charges 50%.

Testing, Limitations, and Future Work

Tested contract paths include happy-path CRUD, validation failures, missing-token 401, wrong-role 403, overlap conflicts 409, and missing-resource 404.

Known limitations: no payment gateway, no email/SMS notifications, no automated admin provisioning, and no production deployment configuration.

Future improvements: transactional booking creation, stronger pagination, audit logs, automated API tests, image uploads, and payment integration.

Conclusion and Viva Walkthrough

The project demonstrates a complete backend workflow rather than isolated CRUD. Every member should explain the request path from route to controller to Mongoose model, the overlap query, the status machine, the inspection charge calculation, and the RBAC decision.

Walkthrough order: authentication and fleet foundation; booking and inspections; reports and policies; infrastructure, validation, and documentation.